Fleet Classes

Porter les règles métier dans des classes Python : `remaining_slots()`, `is_rideable()`, `HelixError`.

---

*Durée indicative : **25 à 35 min** · Organisation : **individuel** · Évaluation :** non noté*

Support : `dock-control/` du TP 3

Contexte

Les routes manipulent encore des dicts. Un dict n'empêche pas `docked` sans `station_id`. Une classe peut porter une **règle**.

- `Station.remaining_slots()` = `capacity - docked_count`
- `Bike.is_rideable()` = statut `docked` **et** `battery_pct >= 15`
- `HelixError` : héritage simple pour nommer le métier (`StationNotFound`, `BikeNotRideable`)

Vous n'enregistrez pas encore ces classes dans une base. Les routes peuvent continuer à servir des dicts **si** les règles sont testées à part (un `if __name__` ou un mini script).

Résultat attendu

1. `app/models.py` définit `Station`, `Bike`, `HelixError` (et au moins une sous-classe).

2. `Station.remaining_slots()` retourne un `int`. Exemple : capacity 12, 9 docked → 3.

3. `Bike.is_rideable()` est `False` si `battery_pct < 15` **ou** si le statut n'est pas `docked`.

4. Lancer `python -c "from app.models import Bike; …"` (ou un petit `scripts/check_models.py`) affiche les deux résultats, sans uvicorn.

5. Aucun « god dict » unique partagé par toutes les règles.

Étapes

Travaillez seul, dans l'IDE.

1. Créer `app/models.py`

Classe `Station` avec au minimum `code: str`, `capacity: int`, `docked_count: int`, méthode `remaining_slots() -> int`.

Classe `Bike` avec `serial: str`, `status: str`, `battery_pct: int`, `station_id: int | None`, méthode `is_rideable() -> bool`.

2. HelixError

```
class HelixError(Exception):
    pass

class StationNotFound(HelixError):
    pass

class BikeNotRideable(HelixError):
    pass
```

Pas d'héritage décoratif de plus. Trois classes suffisent.

3. Script de contrôle

Créez `scripts/check_models.py` qui instancie :

- une station capacity 12, docked 9 → imprime `remaining_slots` (attendu 3)
- un vélo `docked`, `battery_pct=40` → `is_rideable` True
- un vélo `docked`, `battery_pct=10` → False
- un vélo `in_trip`, `battery_pct=80` → False

```
source .venv/bin/activate
```

```
python scripts/check_models.py
```

Les quatre lignes correspondent aux attendus. Si `in_trip` est rideable, la règle est dans la route au lieu de la classe : corrigez la méthode.

4. Vérifier

Relancez le script. uvicorn n'est pas obligatoire pour ce TP. `app/models.py` s'importe sans erreur.

Livrables

- `app/models.py`, `scripts/check_models.py`
- quatre cas visibles dans le terminal
- commit :

```
git add app/models.py scripts/check_models.py
git commit -m "feat(session-01): add station and bike domain classes"
```

Montrez la sortie du script au formateur avant le TP 5.

Suite — TP 05 Open the Dock

Classes et lectures HTTP existent à part. Au TP 5 vous assemblez le squelette : projet 3.12, routes health + stations, classes importées, `/docs` montrable, tag `session-01`.